CATBOT: Reinforcement Learning via Q-learning and Markov

Project Information

Category	Details
Group Name	Malunggay Pandesal
Group Members	Ivan Antonio Alvarez, Bahir Benjamin Barlaan, Vince Miguel Jocson
Course and Section	CSINTSY_S05 (Introduction to Intelligent Systems)

1. MDP Formulation

- **Rewards:** Unlike a standard reinforcement learning environment, this one did not have a default reward system. So what we did was to manually implement a reward system in **training.py** to guide the agent's learning process. The reward system is based on the change in Manhattan Distance between the CatBot and cat after each action.
    - +100 reward is given when the CatBot successfully catches the cat. Otherwise the reward is computed based on the change in the Manhattan Distance. We implemented the reward system in this code block:

```python
# 4. Compute Custom Reward
if terminated:
    reward = 100.0  # Big reward for catching the cat
else:
    # Reward moving closer, penalize moving further away
    distance_change = current_distance - next_distance
    # Small step penalty (-0.1) to encourage speed
    reward = (distance_change * 2.0) - 0.1
```

    - This means that moving closer to the cat results in a positive reward and vice versa. Every move receives a penalty (-0.1) to encourage the CatBot to reach the goal in a fewer amount of steps.
- **Transition Model:** The environment is deterministic. Whenever the CatBot performs an action, it transitions to a new state according to the movement rules of the environment. The next state depends only on the current state and the selected action making the environment suitable for modeling as a Markov Decision Process.

2. Q-Learning Implementation

- **Hyperparameters:**
  - Learning Rate: 0.15
  - Discount Factor: 0.95
  - Initial Epsilon: 1.00
  - Minimum Epsilon: 0.01
  - Epsilon Decay Rate: 0.001
- **Training Episodes: 5,000**

  At first, we tried a learning rate of 0.80 because we assumed that a higher value would make CatBot learn faster. Instead, its Q-values changed too aggressively. We lowered it to 0.50, and a few more other lower values, but it still gave too much importance to the newest experience. A value of 0.10 was more stable but learned slowly, so we finalized it at 0.15 as a balance.

  We initially used a discount factor of 0.50. This worked for Batmeow because catching him only required moving closer, but it was not enough for cats that required several setup moves. We increased it to 0.95 so earlier actions could gain more value from the future +100 catch reward.

  Lastly, the 5,000 episodes were not chosen through testing. This was the project's fixed requirement

- **Learning Process:**

  Each cat was trained separately for 5,000 episodes. At the start of every episode, the environment resets, and CatBot receives its current state. It then chooses between a random action for exploration or the action with the highest Q-value for exploitation.

  After every move, the program calculates the reward based on the change in **Manhattan distance** and updates the Q-table using the Q-learning formula. The exploration rate (Epsilon) gradually decreases, so CatBot starts with trial and error before relying more on what it has learned.

  This process continues until CatBot catches the cat. Since every cat has different behavior, each training run produces a different Q-table even though they all use the same learning process and reward formula.

- **Exploration Strategy:**

We used epsilon-greedy exploration. CatBot begins with an epsilon of 1.00, so it initially explores random actions (Epsilon is 1 cause we encourage it to explore since it has no data/moves known yet). The exploration rate gradually decreases using a decay rate of 0.001 until it reaches 0.01, allowing CatBot to rely more on its learned Q-values without completely stopping exploration.

3. Experimental Results and Performance

A separate script that trains the bot and then runs simulations on each cat 100 times was used to evaluate their performance and make quantitative and qualitative observations.

Cat Name	No. of Fails (More than 60 moves)	Success Rate	Ave. Steps	Min Steps	Max Steps

Batmeow	0	100%	14.00	14	14
Mittens	0	100%	12.70	7	27
Paotsin	0	100%	15.51	14	21
Peekaboo	2	98%	27.70	21	60
Squiddyboi	0	100%	18.93	10	49

Table A: First Evaluation Run

Cat Name	No. of Fails (More than 60 moves)	Success Rate	Ave. Steps	Min Steps	Max Steps
Batmeow	0	100%	14.00	14	14
Mittens	0	100%	12.19	7	23
Paotsin	0	100%	15.33	14	21
Peekaboo	0	100%	27.60	21	50
Squiddyboi	1	99%	18.37	10	60

Table B: Another Evaluation Run

Overall, the training model was quite effective as it gave a 98%-100% success rate across all 5 cats, only having some rare cases where it would take more than 60 moves for Peekaboo and Squiddyboi given their very agile moveset.

Batmeow, Mittens, and Paotsin are all consistent cats that maintain a 100% success rate across all runs and tests, while also all having low average step counts. Runs with Batmeow always learns to take the same number of steps since he just stays in one spot. Mittens has the most spaced out min and max steps since he moves at random, which could make him accidentally move closer or further from the bot.

Given that Peekaboo and Squiddyboi are very dynamic with their moveset and are good at slipping away, they are the cases that saw some failures and outliers. Peekaboo has the highest average steps and often produces fails compared to Squiddyboi because of his phenomenal teleporting prowess. Squiddyboi has lower average steps but still produced some fails in some evaluation runs, but not as often as Peekaboo. His ability to jump over the bot causes the state to flip unexpectedly.

4. Main Challenge encountered

Playing with the cats (play.py):

Playing through play.py helped us understand each cat's weakness. Batmeow was easy because he stays still, while Mittens only moves randomly. The harder cats required specific strategies: Paotsin had to be lured toward a wall, Peekaboo had to be approached from above when he was on the left side of the grid. Our biggest frustration was, "HOW DO YOU EVEN CATCH A CAT THAT JUMPS TWO BLOCKS OVER YOU WHENEVER YOU GET CLOSE?! Seemed impossible (Squiddyboi….)" well after a few runs we understood that Squiddyboi had to jump on us where theres only 1 block space next to us, so that he cant jump 2 blocks and we could catch him.

We knew we could catch them easily using hardcoded conditions, but our problem was: How can CatBot learn these strategies using only one fixed reward formula?

This was where we understood the power of reinforcement learning. Our finished training.py does not contain special rules for walls, corners, jumping, or teleporting. Instead, every cat uses the same reward system: moving closer gives a positive reward, moving farther gives a negative reward, each step has a small penalty, and catching the cat gives +100.

Each cat was trained separately for 5,000 episodes, producing a different Q-table based on its behavior. Through repeated trial and error, actions that eventually led to the +100 reward gained long-term value. This allowed CatBot to learn different strategies without us directly hardcoding each cat's weakness. And in the end this RL process worked.

Another challenge was that env.step() returned a reward of zero. At first, we thought the environment would already tell CatBot whether its action was good or bad. We solved this by creating our own reward using the change in Manhattan distance.

We also had difficulty connecting the theoretical Q-learning formula to the actual code. We eventually understood that obs is the current state, action is the selected movement, and next_obs is the resulting state.

Lastly, all Q-values initially started at zero, so always using np.argmax() would repeatedly select the first action. We solved this using epsilon-greedy selection, allowing CatBot to explore random actions first before slowly relying on what it had learned.

5. Table of Contributions

Features and implementation details were discussed by the group prior to execution.

Task	VINCE	IVAN	BENJAMIN
MDP & Q-Learning	✅	✅	✅
Implementation	✅	✅	✅
Documentation	✅	✅	✅